

Problem Set #8 — Algorithm Report

Aibek Zhazykbek

Problem 1 — SCC and Reversal

1. Algorithm to compute $\text{rev}(G)$ in $O(V + E)$:

Given a directed graph $G = (V, E)$, create a new graph G_{rev} where every edge (u, v) in G becomes (v, u) .

```
for each vertex u in V:  
    G_rev[u] = empty list  
  
for each (u, v) in E:  
    add edge (v, u) to G_rev
```

This visits each vertex and edge once, so complexity is $O(V + E)$.

2. Proof: $\text{scc}(G)$ is acyclic.

Assume for contradiction that the component graph has a cycle $C_1 \rightarrow C_2 \rightarrow \dots \rightarrow C_k \rightarrow C_1$. Then each component can reach the next, and C_k can reach C_1 . Thus all vertices in these components can reach each other, so they must be one single strongly connected component. Contradiction. Therefore, $\text{scc}(G)$ is acyclic.

3. Prove that $\text{scc}(\text{rev}(G)) = \text{rev}(\text{scc}(G))$.

Reversing all edges in G reverses all reachability relations. If two vertices mutually reach each other in G , they also mutually reach each other in $\text{rev}(G)$. Thus SCCs do not change, only the direction of edges between components changes. Hence

$$\text{scc}(\text{rev}(G)) = \text{rev}(\text{scc}(G)).$$

4. Prove: u can reach v in G iff $S(u)$ can reach $S(v)$ in $\text{scc}(G)$.

($\Rightarrow$) If $u \rightarrow v$ in G , then any path from u to v passes through components forming a directed path from $S(u)$ to $S(v)$.

($\Leftarrow$) If $S(u)$ reaches $S(v)$ in component graph, there is a sequence of edges $S(u) \rightarrow C_1 \rightarrow C_2 \rightarrow \dots \rightarrow S(v)$, each corresponding to real edges in G . Inside each component all vertices are mutually reachable. Thus concatenating these, u reaches v .

Problem 2 — Euler Tour

1. **Condition:** A directed graph has an Euler tour iff for every vertex

$$\text{in-degree}(v) = \text{out-degree}(v).$$

Proof:

- In an Euler tour, every time an edge enters a vertex, another edge must exit it. -
Conversely, if degrees match and the graph is strongly connected, Hierholzer's algorithm constructs an Euler cycle.

2. $O(E)$ algorithm (Hierholzer):

1. Start from any vertex.
2. Follow unused edges until returning to start — this forms a cycle.
3. If some vertex in the cycle has unused outgoing edges, start a new cycle there.
4. Merge cycles.

This processes each edge once $\rightarrow O(E)$.

Problem 3 — Topological Sort

Graph edges: $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow C$, $B \rightarrow D$, $C \rightarrow E$, $D \rightarrow E$, $D \rightarrow F$, $G \rightarrow F$, $G \rightarrow E$.

Starting from A: One valid order:

A, B, D, C, G, F, E

Random start example 1:

G, A, B, D, C, F, E

Random start example 2:

A, G, B, C, D, F, E